

Caso pedagógico 03 — Cliente de usuários no auth-service

Contexto

Em arquiteturas de microsserviços, o **auth-service** raramente é “isolado”: ele depende de serviços de usuários para consultar perfis, verificar status, validar credenciais, recuperar permissões, entre outros. Por isso, a forma como o **cliente HTTP interno** é implementado (URL, timeouts, retries, autenticação de serviço, padronização de headers) influencia diretamente a **disponibilidade do sistema**.

Este caso pedagógico mostra como um cliente HTTP mal “hardenizado” pode gerar instabilidade e riscos de segurança — e como corrigir com princípios simples: **consistência**, **resiliência** e **fail-fast**.

Problema identificado

O cliente HTTP que o `auth-service` usa para chamar o `users-service` apresentava três fragilidades principais:

1. **Fallback inconsistente de URL**
 2. Em alguns cenários, o cliente caía para `http://users:3000`.
 3. Isso cria risco de **endpoint inválido** em produção (nome de serviço, porta e DNS interno diferentes do esperado).
 4. **Ausência de timeout e política de retry**
 5. Sem timeout, requisições podem ficar “presas” aguardando resposta, consumindo recursos do `auth-service`.
 6. Sem retry com backoff, pequenas instabilidades (restart, pico de latência) derrubam a operação.
 7. **Headers montados a cada chamada e `SERVICE_TOKEN` sem validação**
 8. O token de serviço não era exigido no momento de construção.
 9. Isso abre margem para chamadas internas **sem autenticação de serviço** (ou com token vazio), aumentando risco de acesso indevido e dificultando auditoria.
-

Impacto potencial

1. **Indisponibilidade do auth-service por efeito dominó**
2. Se o `users-service` reinicia ou responde lentamente, o `auth-service` pode saturar suas conexões/threads e tornar-se indisponível.
3. **Chamadas internas sem token de serviço (ou token inválido)**

4. A ausência de `fail-fast` permite que requisições saiam sem `x-service-token`, gerando falhas intermitentes e risco de segurança.

5. Risco de endpoints errados em produção

6. Um fallback para `users:3000` pode funcionar “por acaso” em um ambiente e quebrar em outro — o pior tipo de bug: o que aparece só na hora do deploy.

Solução fundamentada

A correção adotou três princípios operacionais claros:

1) Consistência de endereço interno (DNS + porta)

Remover fallback e **padronizar a URL interna** do `users-service` no Compose: - `http://users-service:3011`

Isso usa o DNS interno do Docker/Compose (nome do serviço) e a porta interna esperada pelo container.

2) Resiliência por padrão (timeout + retries com backoff)

Criar **uma única instância do Axios** configurada com: - timeout **5s** - retries **3 tentativas** - backoff exponencial (ex.: 200ms → 400ms → 800ms)

Com isso, pequenas oscilações do `users-service` deixam de derrubar o `auth-service`.

3) Fail-fast para autenticação de serviço

Exigir `SERVICE_TOKEN` na construção do client: - se estiver ausente/vazio, **falhar imediatamente** no bootstrap.

Além disso: - aplicar `x-service-token` como header padrão do cliente (não a cada chamada) - garantir que **todas** as requisições internas carreguem o token.

Implementação (delta aplicado)

1) Ambiente (prod/deploy)

Os arquivos já estavam alinhados para usar: - `.env.prod` e compose (prod/deploy): - `USERS_API_URL=http://users-service:3011`

Esse valor elimina a necessidade de fallback e padroniza o endereço interno.

2) Código — `packages/auth-service/src/users-http.client.ts`

Mudanças-chave: - Base URL consistente (sem fallback): usa `USERS_API_URL` apontando para `http://users-service:3011`. - Cria **client Axios único**: - timeout: **5s** - retries: **3 tentativas** - backoff exponencial - Exige `SERVICE_TOKEN` no construtor (fail-fast). - Define `x-service-token` como header padrão, reaproveitado em todas as chamadas.

Efeito prático (o que muda no comportamento do sistema)

Após a correção: - O `auth-service` chama o `users-service` por um endereço interno **consistente**. - As chamadas passam a ser **mais resilientes** a latência e reinícios do `users-service`. - Se o token de serviço estiver ausente, o sistema falha de forma **explícita e imediata** (melhor do que falhas intermitentes em runtime).

Checklist didático para validação em aula

1. `USERS_API_URL` em produção aponta para `http://users-service:3011`.
 2. O cliente HTTP não possui fallback para `users:3000`.
 3. Existe uma instância Axios única (singleton) usada pelo módulo/serviço.
 4. Timeout configurado (5s) e retries com backoff (3 tentativas).
 5. `SERVICE_TOKEN` é obrigatório e validado no bootstrap.
 6. Header `x-service-token` é aplicado em todas as requisições internas.
-

Perguntas para discussão (para consolidar aprendizado)

1. Por que um fallback de URL pode parecer útil em dev, mas ser perigoso em produção?
 2. O que acontece com um serviço sem timeout quando o upstream está lento?
 3. Em quais casos retries ajudam e em quais podem piorar o cenário?
 4. Por que "fail-fast" (falhar no bootstrap) é preferível a falhas aleatórias em runtime?
 5. Que tipos de chamadas são seguras para retry automático? (idempotentes vs não-idempotentes)
-

Próximas melhorias (opcionais)

- Adicionar logs/métricas por tentativa (quantidade de retries, tempo total, taxa de erro).
 - Validar contrato de resposta do `users-service` com DTO/schema (ex.: Zod/Class Validator), para reduzir "quebras silenciosas".
-

Conclusão

Este caso reforça um ponto-chave em microsserviços: **a confiabilidade do sistema depende da confiabilidade das integrações.**

Ao remover fallback, padronizar baseURL, introduzir timeout/retries e exigir token de serviço com fail-fast, o `auth-service` passa a operar de forma mais previsível, segura e resiliente — com menor risco de instabilidade em produção.